

Software Requirement Specification (SRS) Document

Project Name: CampusCode - College Coding & Aptitude Practice Platform

Date: 29-07-2025

Version: 1.0

By: Team Innovaid

Revision History

Version	Author	Comments	Date Completed
1.0	Team Innovaid	Initial Draft	29-07-2025

Approval/Reviewer History

Reviewer	Version Reviewed	Signature	Date
----------	------------------	-----------	------

Table of Contents

1. Introduction
2. Functional Requirements
3. External Interface Requirements
4. Non-Functional Requirements
5. Definitions and Acronyms

1. Introduction

1.1 Product Scope

CampusCode is a web-based coding and aptitude practice platform tailored for college use. It allows students to solve programming and aptitude problems, attend assignments/tests, and allows teachers to assign, track, and analyze student progress.

1.2 Product Value

It offers an all-in-one educational environment, reducing dependency on multiple platforms. It builds coding culture within colleges and supports teacher-student engagement through task monitoring, automated test grading, and performance tracking.

1.3 Intended Audience

- College Students
- Professors / Instructors
- College Admins

1.4 Intended Use

- Students use it to solve problems and attend tests.
- Teachers assign problems and monitor progress.
- Admins manage users and oversee operations.

1.5 General Description

The software will have:

- Role-based login (Student, Teacher, Admin)
- Dashboard for solving coding/aptitude problems
- Test & assignment modules
- Real-time reports & analytics
- Secure and scalable architecture

2. Functional Requirements

- Student registration/login
- Solve problems (coding + aptitude)
- Teachers create/assign tests & homework
- Timer-based test environment
- Code execution using APIs (e.g., Judge0)
- Plagiarism detection for code
- Performance tracking & analytics
- Admin to manage users and settings

3. External Interface Requirements

3.1 User Interface Requirements

- Clean dashboard (per role)
- IDE-like code editor
- MCQ-based quiz interface
- Responsive UI for mobile/desktop

3.2 Hardware Interface Requirements

- Accessible on browsers (Chrome, Edge, Firefox)
- No specific hardware required

3.3 Technology Specification

- Frontend: React.js / Next.js
- Backend: Node.js with Express / Django
- Database: Firestore / MongoDB / PostgreSQL
- Auth: Firebase / JWT
- Hosting: Vercel / Firebase / Render

3.4 Communication Interface Requirements

- HTTPS protocol
- RESTful API communication between frontend/backend
- Email for test notifications and account recovery

4. Non-Functional Requirements

4.1 Security

- Secure authentication & authorization
- HTTPS encryption
- Anti-cheat/plagiarism control

4.2 Performance Requirements

- Support 500+ concurrent users
- Code execution time: <3 seconds average

4.3 Compatibility

- Compatible with major browsers and devices
- Responsive design for mobile/tablet/desktop

4.4 Reliability

- 99.9% uptime expected
- Fallback systems for code API failures

4.5 Scalability

- Easy to onboard new colleges
- Multi-tenant architecture support

4.6 Maintainability

- Modular architecture
- CI/CD support
- Detailed documentation for devs

4.7 Usability

- Minimal learning curve
- Clean and intuitive user interface
- Gamification elements (badges, streaks)

4.8 Other Requirements

- Admin usage logs
- Feedback collection system

5. Definitions and Acronyms

- SRS: Software Requirements Specification
- MVP: Minimum Viable Product
- MCQ: Multiple Choice Questions
- IDE: Integrated Development Environment
- CRUD: Create, Read, Update, Delete
- API: Application Programming Interface
- JWT: JSON Web Token
- CI/CD: Continuous Integration / Continuous Deployment